

NAME - HARSH RANJAN
ROLL - UG104/BTCSE/2025/069
SEC - A

PAGE NO.

Date: / /

linked list test

1. Ans → (b) Data and pointer to next node
2. Ans → (c) NULL
3. Ans → (b) self-referential structure
4. Ans → (b) head = NULL
5. Ans → (b) newnode → next = head; head = newnode;
6. Ans → (b) temp = head; head = head → next free(temp);
7. Ans → (c) Traversing from tail to head.
8. Ans → (b) n
9. Ans → (c) Copying the data of p → next into p, then deleting p → next.

10. Ans → (b) while (p != NULL)

HT

NAME - HARSH RANJAN
ROLL - UG104/BTCSE/2025/069
SEC - A

11. Ans → (b) p → next = NULL
12. Ans →
13. Ans → (c) Three
14. Ans → (b) Exactly one node
15. Ans → (b) (struct node*) malloc (size of (struct node))
16. Ans → (b) Avoid special-case handling for insertion and deletion
17. Ans → (c) The first node
18. Ans → (b) It does not support random (linked list) access
19. Ans → (b) Loss of access to all
20. Ans → (b) A slow pointer and one fast pointer
21. Ans → (c) The last node
22. Ans → (b) Nodes may be scattered anywhere in memory
23. Ans → struct Node {

int data;
struct Node * next;

};

NAME - HARSH RANJAN
ROLL - UG104/BTCSE/2025/069

Teacher's Signature.....

24. Ans → 30, 40, 70

28 (i) Ans → struct node * Insert At Beginning
(struct node * head, int data) {

struct node * ptr = (struct node *) malloc
(size of (struct node));

ptr → next = head;
ptr → data = data;

return ptr;

}

head = insert At beginning (head, 7);
linked list Traversal (head);

return 0;

}

(iii) Ans →

struct node * Insert at End (struct node * head,
int data) {

struct node * ptr = (struct node *) malloc (size of
(struct node));

struct node * p = head;

p = p → next;

while (p → next != null) {

}

p = p → next

p → next = ptr;

ptr → next = NULL;

ptr → data = data;

return head;

}

NAME = HARSH RANJAN

EL ROLL = UG/04/BTCSE/
2025/069

SEC = A

Teacher's Signature.....

head = InsertAtend (head, 9);
linked list Traversal (head);

return 0;
}

NAME = HARSH RANJAN

ROLL = UG/04/BTCSE/2015/
069

SEC = A